A 28-nm 28.8-TOPS/W Attention-Based NN Processor With Correlative CIM Ring Architecture and Dataflow-Reshaped Digital-Assisted CIM Array

Ruiqi Guo, *Graduate Student Member, IEEE*, Zhiheng Yue, Yang Wang, Hao Li, Te Hu, Yabing Wang, Hao Sun, Jeng-Long Hsu, Yaojun Zhang, Bonan Yan, *Member, IEEE*, Leibo Liu, *Senior Member, IEEE*, Ru Huang, Shaojun Wei, *Fellow, IEEE*, and Shouyi Yin, *Senior Member, IEEE*

***Abstract*— Transformer models have achieved impressive performance in various applications by effectively capturing contextual knowledge from the entire sequence. However, the multi-headed self-attention (MHSA) mechanism of Transformer models introduces multiple rounds of matrix multiplication (MM) and Softmax operations, which results in massive data movement and computations. Compute-in-memory (CIM) is a promising candidate to reduce data movement in the memory hierarchy of artificial intelligence (AI) accelerators, increasing the speed and energy efficiency for MM computation. However, the attention mechanism introduces dynamic MMs involving *Query* (*Q*), *Key* (*K*), and *Value* (*V*). Since these matrices are both generated dynamically in previous layers, the dynamic MM mismatches the CIM paradigm, resulting in significant energy/latency consumption. This article proposes a CIM-based transformer accelerator (TranCIM) with three design features, effectively handling dynamic MMs. First, a correlative CIM ring (CRCIMR) executes the dynamic MM that involving *Q* and *K* by matrix decomposition, removing the loading of dynamically generated matrix in SRAM-based CIM (SRAM-CIM) cells. Second, a Softmax-based speculation unit (SSU) reduces the computation redundancy in dynamic MMs. Third, a digital-assisted CIM array (DACIMA) executes the dynamic MM that involving *V* based on symmetrical-L-shaped products, allowing the CIM macro to work in compute mode continuously. Fabricated in a 28-nm CMOS technology, the proposed accelerator occupies an area of 7.08 mm^2. Measured on TinyBERT and BERT-Base with INT8 precision, the proposed accelerator achieves a system-level energy efficiency of 28.8 TOPS/W.**

***Index Terms*— Compute-in-memory (CIM), pipeline, processor, self-attention, sparsity, speculating, transformer.**

Manuscript received 29 November 2023; revised 15 February 2024, 17 April 2024, and 19 June 2024; accepted 19 June 2024. Date of publication 2 August 2024; date of current version 20 January 2025. This article was approved by Associate Editor Ben Keller. This work was supported in part by the National Key Research and Development Project under Grant 2023YFB4403100, in part by NSFC under Grant 62125403 and Grant 92164301, in part by the National Science and Technology Major Project under Grant 2022ZD0115201, in part by the National Key Research and Development Program under Grant 2021ZD0114400, in part by Beijing S&T Project under Grant Z221100007722023, in part by Beijing National Research Center for Information Science and Technology, and in part by Beijing Advanced Innovation Center for Integrated Circuits. *(Corresponding author: Shouyi Yin.)*

Ruiqi Guo, Zhiheng Yue, Yang Wang, Hao Li, Te Hu, Yabing Wang, Hao Sun, Leibo Liu, Shaojun Wei, and Shouyi Yin are with the School of Integrated Circuits, Beijing National Research Center for Information Science and Technology, and Beijing Advanced Innovation Center, Tsinghua University, Beijing 100084, China (e-mail: yinsy@tsinghua.edu.cn).

Jeng-Long Hsu is with NeoNexus Pte. Ltd., Singapore 079903.

Yaojun Zhang is with Pimchip Technology Company Ltd., Beijing 100084, China.

Bonan Yan and Ru Huang are with the Institute for Artificial Intelligence and Beijing Advanced Innovation Center for Integrated Circuits, School of Integrated Circuits, Peking University, Beijing 100871, China.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSSC.2024.3419808.

Digital Object Identifier 10.1109/JSSC.2024.3419808

I. Introduction

TRANSFORMER models are a type of deep neural network (DNN) architecture that has revolutionized the field of natural language processing (NLP) in recent years [1]. In contrast to recurrent neural networks (RNNs) and convolution neural networks (CNNs), which process input sequentially or with fixed-size filters, Transformer models utilize a multi-headed self-attention (MHSA) mechanism to capture contextual knowledge from the entire input sequence. Although large Transformer models outperform CNNs in terms of accuracy, they introduce a large number of computations due to its processing of all input tokens. Specially, GPT-2 requires 91.2% more computations than RNNs [2], and the operation amount of Swin-B is 94.8× higher than EfficientDet [3]. The intensive computation severely prevents their deployment for a good ecosystem.

The key of compute-in-memory (CIM) is to combine memory cells and multiply-and-accumulate (MAC) logic. Embedding computations inside memory has three significant benefits to improve compute efficiency. The benefits are particularly evident when performing a large number of matrix multiplications (MMs) with the stored weight matrix in Transformer models. First, data transfer to/from the memory is greatly reduced since weight matrices are not explicitly read and only the computed output is sent outside the memory. Second, we can take advantage of the massively parallel nature of Transformers to reuse the loaded weight matrix for multiple input vectors' computations. Third, CIM allows parallel MAC operations by activating multiple memory addresses simultaneously. Furthermore, SRAM-based CIM (SRAM-CIM) is a promising candidate to accelerate Transformer models due to its fast read/write speed and good compatibility with standard CMOS fabrication processes [4], [5], [6], [7], [8], [9].

Unfortunately, the MHSA mechanism introduces new challenges for previous CIM-based accelerators, as MHSA

0018-9200 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

involves dynamic MMs and multiple rounds of Softmax activation. One kind of dynamic MM involves *Query (Q)* and *Key (K)*, and the other involves *Possibility (P)* and *Value (V)*, both of which are the computational bottleneck of the MHSA layer. Taking the dynamic MMs of Q and transposed K as an example, Q and K are both generated in runtime. In the traditional manner, *Matrix K* has to be written out and then loaded back to CIM, resulting in redundant latency of CIM reloads. Furthermore, the dynamic MM between Q and K contains substantial redundancies since many products are activated to be zero-valued elements of *Matrix P* through the Softmax function, which are ineffectual inputs for the dynamic MM of P and V. Computations involving these zero-valued possibilities are redundant for the current dynamic MM between Q and K, resulting in a waste of time and energy.

Recently, a CIM-based Transformer accelerator, called TranCIM [10], also discusses the issue of dynamic MMs. To avoid redundant off-chip memory access, TranCIM connects its CIM engines in a two-stage pipeline to perform dynamic MMs. The first stage's CIM macros perform static MMs with weight matrix and generate the intermediate matrices Q and K. The two dynamically generated matrices are sent to the second stage's CIM macros for subsequent computations of $Q \times K^{\mathrm{T}}$. However, TranCIM typically exists a significant number of pipeline bubbles and introduces additional memory access energy consumption due to three reasons: 1) matrix K needs to be entirely stored in CIM macros before conducting the MAC operations of $Q \times K^{\mathrm{T}}$; 2) there exists a considerable bandwidth gap between massively parallel computing and rowwise memory access in CIM macros; and 3) writing matrix K into SRAM-CIMs leads to a lot of data movement per MAC operation.

This article proposes a Transformer CIM accelerator, called AttCIM [11], using analog CIM macros as computing units. AttCIM efficiently handles the two types of dynamic MMs and reduces redundant computations with three architectural innovations, outlined in the following.

1) We propose a correlative CIM ring (CRCIMR) architecture to execute the dynamic MM that involves Q and K through matrix decomposition, removing the loading of dynamically generated matrix in CIM macros.
2) We design a Softmax-based speculation unit (SSU) to dynamically capture redundant attention connections by leveraging the property of the Softmax function, reducing redundancies of dynamic MM computations.
3) We propose a digital-assisted CIM array (DACIMA) to execute the dynamic MM that involves P and V through matrix decomposition. The DACIMA performs the decomposed $P \times V$ based on sparse L-shaped products, allowing the CIM macro to work continuously in the compute mode.

The rest of this article is organized as follows. Section II describes the background and motivation of this work. Section III presents AttCIM's overall architecture. The three architecture techniques are, respectively, detailed in Sections IV–VI. Section VII shows chip implementation and extensive measurement results. Section VIII concludes this article.

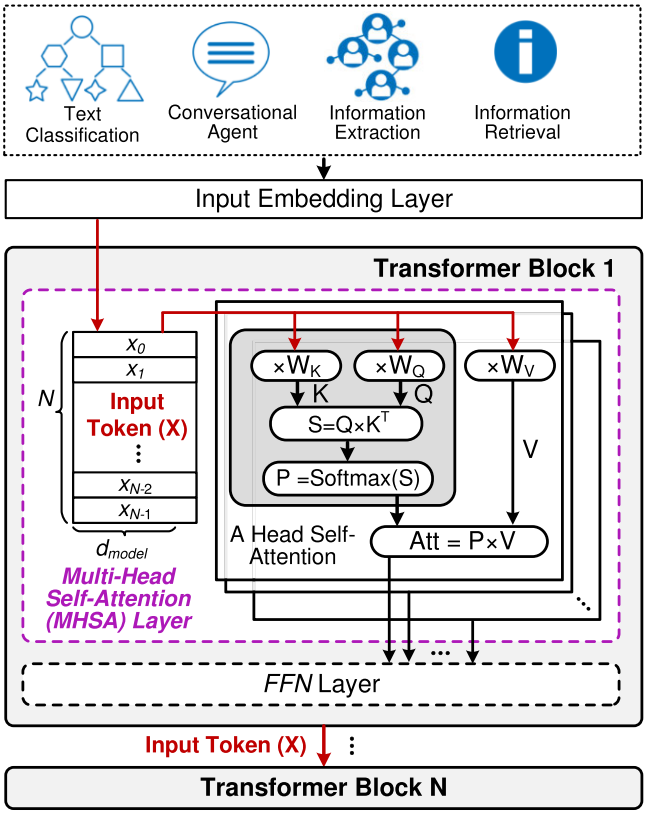


Fig. 1. Transformer model with MHSA mechanism.

II. Background and Motivation

A. MHSA Mechanism

Fig. 1 shows the components of a Transformer model, which comprises several identical Transformer blocks connected in sequence. Each Transformer block consists of an MHSA layer and a feed-forward network (FFN). In each Transformer block, the embedding words in the sequence, called input tokens, are processed through an MHSA layer and fed into an FFN to obtain a new representation. This new representation is then passed to the next Transformer block for further processing, and this process is iterated until the output of the last Transformer block is obtained. The new representation, commonly referred to as the input token of each Transformer block, is typically denoted as *Matrix X* with a size of $N \times d_{\mathrm{model}}$, where N represents the token length and d_{model} represents the dimension of each token. In each attention head, the operation is defined as

$$A = \mathrm{Softmax}(\alpha Q \times K) \times V \tag{1}$$

where $Q = X \times W_Q$, $K = X \times W_K$, and $V = X \times W_V$. W_Q, W_K, and $W_V \in \mathbb{R}^{d_{\mathrm{model}} \times (d_{\mathrm{model}}/H)}$ are the weight matrices for the Q, K, and V, which project the input X to an output tensor of d_{model} dimensions. H is the number of heads, and α is a scaling constant. *Matrix Q* and the transposed K are multiplied to compute the *Matrix Score* $S = Q \times K^{\mathrm{T}}$. Subsequently, the Softmax function is applied to each row of S, which normalizes the scores into a vector of P. Finally, *Matrix P* is multiplied by *Matrix V* to compute the attention layer's output matrix $A = P \times V$. The attention outputs from all heads are

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

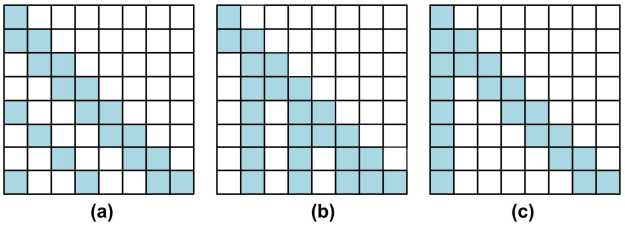


Fig. 2. Typical sparse attention pattern (dense-blue, sparse-white). (a) Stride local. (b) Fixed local. (c) Global + Local.

then passed to an FFN, which consists of two fully connected layers.

In this work, our emphasis is on accelerating the MHSA layer rather than the FFN for two reasons. First, the MHSA layer dominates the Transformer's computations in large language models due to the complexity of $O(N^2)$ in $Q \times K^{\mathrm{T}}$ and $P \times V$, while the computation of the FFN is linearly related to N. Second, previous CIM chips have optimized the FFN [7], [8], [12], in which Guo et al. [12] apply tensor-train decomposition (TTD) to compress and store heavy-weighted FFNs into SRAM-CIM entirely, thereby achieving extremely high energy efficiency. As transformer-based language models can also be decomposed and compressed by TTD [13], the decomposed FFNs can also be optimized by Guo et al. [12] to improve energy efficiency.

B. CIM and Local Sparse Attention

By offloading computation from centralized processing units, CIM dramatically eliminates the frequent data movement between memory and processing units and thus forms an energy-efficient computing system. In particular, a 28-nm 64-kb SRAM-CIM macro only consumes 173.9 fJ for an 8-bit MAC operation [4]. For a new MM in this SRAM-CIM macro, the weight matrix must be pre-written into bitcell arrays. On the contrast, the weight matrix is only loaded into registers in a conventional systolic array. The energy consumption of memory access in SRAM is several times higher than that of accessing registers. Fortunately, the weight matrix will be multiplied by large amounts of input tokens due to the massively parallel nature of Transformer models. With an increasing number of input tokens, the extensive reuse of weights substantially mitigates the energy consumption incurred by CIM reloads. We, respectively, evaluate the energy consumption per operation in the SRAM-CIM macro and a 28-nm conventional systolic array at 0.9 V and 200 MHz. When processing an input sequence of 512 tokens, the cost of loading weights is only equivalent to 0.6% of computing energy consumption. Consequently, SRAM-CIM obtains 2.95× energy efficiency compared to conventional digital architecture. This improvement is obtained without including normal SRAM buffers, which only consider MAC operations and weight reloads in CIM macros.

MHSA layers dominate the Transformer's computation due to the complexity of $O(N^2)$. To reduce the quadratic complexity, many sparse Transformers have been proposed by leveraging local attention mechanism, in which each token can attend to only a subset of the entire tokens [14], [15], [16], [17], [18], [19], [20]. Fig. 2 shows three typical sparse

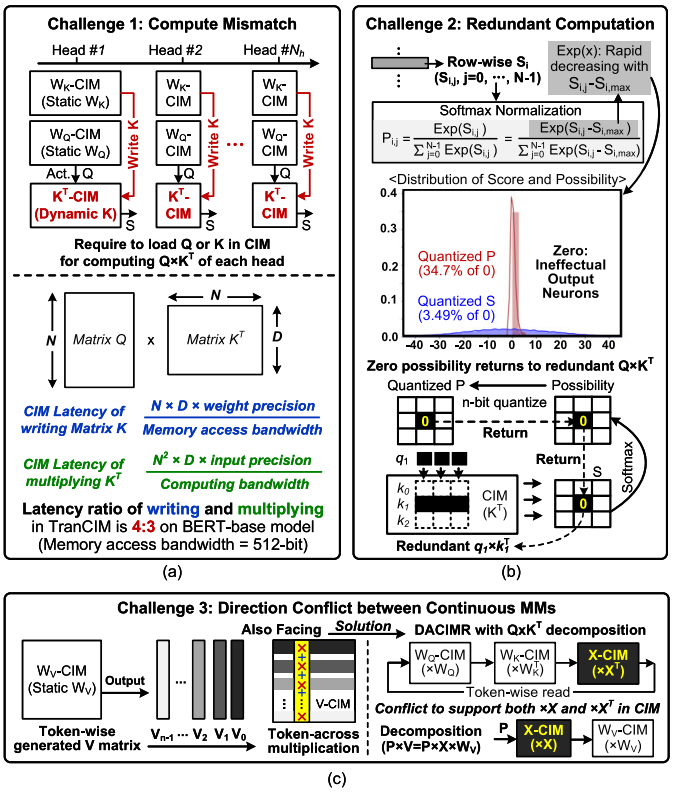


Fig. 3. CIM architecture faces three challenges for accelerating transformer model.

attention patterns. The strided pattern attends every lth location, where lth is the stride step. The fixed pattern attends to specific positions defined by the algorithm. Although slightly different, the key of sparse attention is to limit the attention span to neighboring tokens, thereby reducing the complexity to $O(N)$.

C. Challenges of CIM-Based Chips for MHSA Layers

1) Dominant Computations Mismatch CIM Paradigm: Traditional CNNs are composed of static MMs, with pre-trained weights that remain constant during inference. These static weights can be pre-loaded into CIM macros and fully reused in a weight-stationary dataflow. However, in MHSA layers, the operands of $Q \times K^{\mathrm{T}}$ are dynamically generated from linear transformations. This necessitates the dynamic writing of *Matrix* Q or K into CIM macros as memory data before computing each attention head. As a result, the Transformer's dominant computations do not match the CIM paradigm. Fig. 3(a) illustrates this mismatch in a recent TranCIM [10]. The benchmark involves $Q \times K^{\mathrm{T}}$ with INT8 precision, $N = 2048$, and $d = 512$. Assuming a 512-bit memory access bandwidth, TranCIM incurs over 57% latency to load *Matrix* K in CIM macros during the computation of $Q \times K^{\mathrm{T}}$. When considering Q, K generation, $Q \times K^{\mathrm{T}}$ constitutes 66.7% of computations, with data loading accounting for 88.9% of the latency.

2) Redundant Computation in CIM: The Softmax function exponentially scales up and down for the large and small attention scores, respectively, and generates *Matrix* P. The value in ith row and jth column of *Matrix* P indicates the correlation between token i to token j. A low probability signifies a weak

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

contextual relationship between the two tokens. We observe that over 33.3% attention probabilities are exactly zero after quantization, which is 10× more than zero-valued attention scores. The zero-valued probability corresponds to ineffectual attention computations, having no impact on subsequent $P \times V$ and being redundant for the current $Q \times K^{\mathrm{T}}$. For example, in Fig. 3(b), if $P_{1,1} = 0$, then the computation of $q_1 \times k_1^{\mathrm{T}}$ is redundant.

3) Opposite Computation Direction in CIM: The computation of $Q \times K^{\mathrm{T}}$ and $P \times V$ has two key commonalities: 1) all operands are generated in runtime and 2) K and V result from the linear transformation of X. However, while V can undergo direct multiplication, K needs transposition before multiplication. This discrepancy leads to a hardware conflict in supporting $Q \times K^{\mathrm{T}}$ and $P \times V$ for TranCIM [10]. To avoid introducing additional transpose buffers, TranCIM stores K in a transposed way and aligns the input (Q) feeding and weight (K) writing directions. However, TranCIM only focuses on optimizing $Q \times K^{\mathrm{T}}$, ignoring the computation direction conflict between $Q \times K^{\mathrm{T}}$ and $P \times V$. In the opposite computation direction of K, a column of V is multiplied simultaneously. Consequently, TranCIM still requires to write out V and then load it back in the transpose way when processing $P \times V$, thereby resulting in high energy consumption and latency. Similarly, although the CRCIMR optimizes $Q \times K^{\mathrm{T}}$ by matrix decomposition, the integrated CIM macros (X-CIM) that store X also encounter the computation direction conflict when processing the decomposed $P \times V$. In particular, as shown in Fig. 3(c), the X-CIM requires to support multiplying X and X^{T}, which mismatches the single MAC direction.

III. AttCIM's Overall Architecture

This section first details the fundamental computing unit (SRAM-CIM macro) in AttCIM. Subsequently, we present an overview of the SRAM-CIM Transformer accelerator.

A. SRAM-CIM Macro

Fig. 4(a) shows the structure of the 64-kbit SRAM-CIM macro, consisting of two homogeneous banks. The CIM bank is similar to the design from [4], which comprises 32 parallel weight-bitwise compute-arrays (WBCAs), 32 groups of double 4-bit analog-to-digital converters (ADCs), a digital weight-configuration circuit (DWCC), input drivers, and other peripheral circuits. Considering that the Transformer model incorporates various nonlinear activation functions, such as Softmax, SiLU [31], GELU [32], *etc.*, it becomes sensitive to the accumulation error of partial sums. To ensure the precision of the partial sums computed by CIM macros, each WBCA conducts 16 current-based MAC operations with 2-bit inputs and 1-bit weights in each cycle, thereby aligning with the ADC precision. Each group of ADCs digitizes two analog MAC results and outputs them to the DWCC. The DWCC comprises digital shifts and a 64-input adder tree, which combines bitwise MAC results based on weight precision. For INT8 precision, individual weights are divided by bits and stored in eight adjacent WBCAs. Subsequently, each CIM bank produces four 14-bit MAC results, which undergo

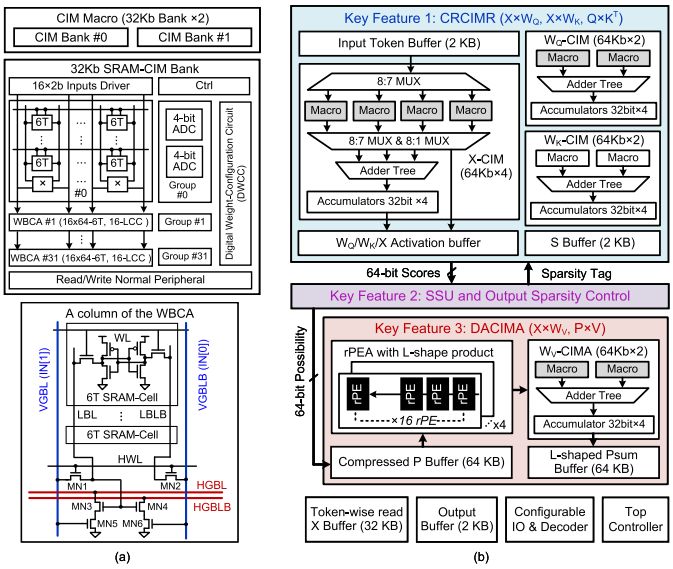


Fig. 4. Overall architecture. (a) Structure of SRAM-CIM macro. (b) System-level overall architecture.

shifting and accumulation outside the CIM macro to handle 8-bit activations.

Each WBCA contains 16 × 64.6T SRAM cells and 16 local computing cells (LCCs). The LCC performs the multiplication of 2-bit unsigned input and 1-bit weight. During computations, the local bitline (LBL) reflects a 1-b weight (W[0]) stored in an accessed SRAM cell, and controls the MN3/MN4. In each cycle, a 2-b input (IN[1:0]) is split by bit and, respectively, applied to a pair of vertical global bitlines (VGBL/VGBLB). The N3–N5 pair then multiplies the IN[0] with W[0] and outputs the result to the horizontal global BL (HGBL). Simultaneously, the N4–N6 pair outputs the product of IN[1] and W[0] to HGBLB.

B. Overall Architecture of AttCIM

Fig. 4(b) shows the overall architecture of AttCIM with three key features. AttCIM is composed of a CRCIMR, an SSU, a DACIMA, a memory system, and a top controller for instruction decoding. The memory system consists of dedicated buffers to store input tokens, partial accumulated results, and final computation results of attention scores and probabilities. These buffers are physically separated and interlinked between different processing modules.

1) CRCIMR: The CRCIMR consists of eight 64-kb SRAM-CIM macros, merging the MMs of X with W_Q, X with W_K, and Q with K^{T} through a CIM-matched manner. In CRCIMR, $Q \times K^{\mathrm{T}}$ is computed by decomposing it into three cascaded MMs as $X \times W_Q \times W_K^{\mathrm{T}} \times X^{\mathrm{T}}$. As a result, the CRCIMR eliminates the need to load *Matrix K* into CIM macros during runtime. Typically, these integrated CIM macros are allocated in groups of 4, 2, and 2 to store *Matrix X*, W_Q, and W_K, respectively. This configuration forms three CIM macro arrays, including the X-CIM array, W_Q-CIM array, and W_K-CIM array. In CRCIMR, taking inputs from the X-CIM, the first two cascaded MMs are performed in the W_Q-CIM and W_K-CIM arrays, respectively. The results of $X \times W_Q \times W_K^{\mathrm{T}}$ are quantized and then returned to the X-CIM macros as inputs for the multiplication of X^{T}.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

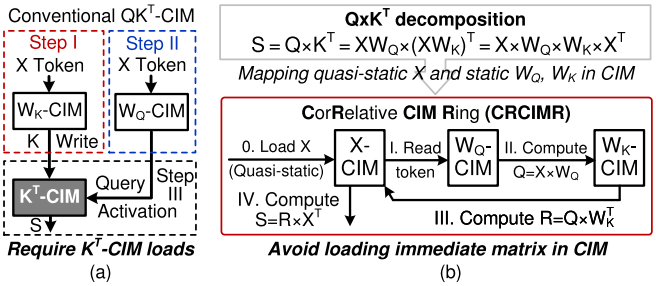


Fig. 5. Using CIM architecture to compute S. (a) Conventional QK^{T}-CIM architecture like [10]. (b) CRCIMR with $Q \times K^{\mathrm{T}}$ decomposition.

2) SSU: The SSU receives attention scores (S) from the CRCIMR, efficiently executing Softmax function to generate the attention probability (P). Furthermore, the SSU speculates the widely distributed output sparsity of P, thereby controlling the CRCIMR to eliminate redundant computing of $Q \times K^{\mathrm{T}}$ by the resulting sparsity index. In a cycle, the CRCIMR either takes 2-bit token data as activations in $X \times W_Q$, or takes 1-bit token data as weights in the multiplication of X^{T}. Therefore, the high bits of the attention scores involving high bits of X (X_{HBs}) are generated in advance. The SSU captures dynamic sparsity of P based on the operations of X_{HBs}. The resulting sparsity index is used to enable the operation of low bits of X (X_{LBs}).

3) DACIMA: The DACIMA receives P from the SSU and computes $P \times V$ in fully pipeline. The DACIMA consists of a reconfigurable processing element array (rPEA) and two CIM macros storing W_V. In the DACIMA, $P \times V$ is decomposed into two cascaded MMs as $P \times X \times W_V$. The rPEA computes $P \times X$ based on L-shaped dot-products, which only uses the currently read tokens in the processing of $X \times W_Q$. The rowwise generated $P \times X$ are sent to W_V-CIM, thereby obtaining the final result of an attention head.

IV. CRCIMR Architecture

A. Correlative Computation Mapping

Fig. 5(a) shows the conventional CIM-based architecture that generates *Matrix S* through three steps. Before executing $Q \times K^{\mathrm{T}}$ (Step III), *Matrix K* that obtained by multiplying X with W_K (Step I) has to be loaded into the CIM cells as weights. Instead, the CRCIMR utilizes a series of quasi-static matrices (including *Matrix X*, W_Q, and W_K) to avoid loading the dynamically generated matrix. In the aspect of algorithm, the MM $Q \times K^{\mathrm{T}}$ can be decomposed into two cascaded MMs as follows:

$$S = Q \times K^{\mathrm{T}} = Q \times (X \times W_K)^{\mathrm{T}} = \left(Q \times W_K^{\mathrm{T}}\right) \times X^{\mathrm{T}}. \quad (2)$$

Fig. 5(b) shows that the decomposition provides an opportunity to avoid the loading of the intermediate matrix into CIM macros as follows. Before computing, three CIM arrays are initialized to store X, W_Q, and W_K^{T} matrices as X-CIM, W_Q-CIM, and W_K-CIM, respectively. In the first compute step (Step I), a row of X (X_i) is read from X-CIM and sent to W_Q-CIM as activations. In the subsequent two step, the CRCIMR performs $Q_i = X \times W_Q$ on the W_Q-CIM and $R_i = Q_i \times W_K^{\mathrm{T}}$ on the W_K-CIM. Finally, X-CIM receives R_i and generates a row of attention score (S_i) by performing $S_i = R_i \times X^{\mathrm{T}}$.

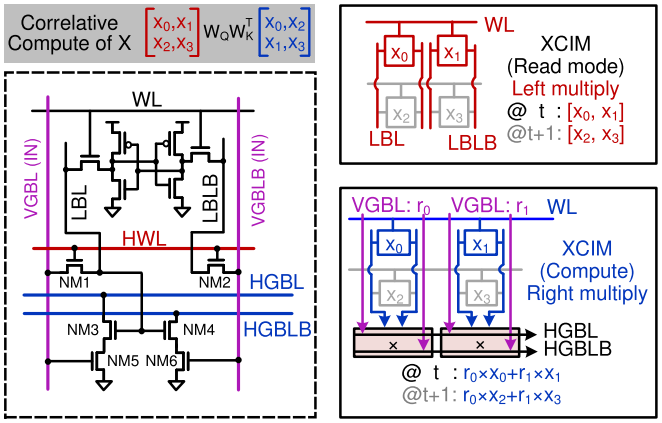


Fig. 6. Concept of correlative mapping in CIM.

In CRCIMR, each CIM macro has three operating modes, including Read, Write, and CIM mode. Similar to previous CIM chips, W_Q-CIM and W_K-CIM work in Write mode and CIM mode, respectively, loading weights and performing MAC operations with activated weights. To multiply Q with decomposed K^{T}, X-CIM needs to work in Read mode and CIM mode, where the stored tokens are, respectively, taken as activations and weights to generate Q and S, respectively.

The computation involving X is similar to the correlative computation of vectors in statistics. Fig. 6 shows the mapping strategy, called correlative mapping, that aligns the activation and weight access directions in X-CIM simultaneously. The elements in a row of X are stored along the wordline (WL) direction. With activating horizontal wordline (HWL), through LBL, LBLB, the two pass gates (MN1/MN2 with HWL = 1), VGBL and VGBLB, a token's elements can be read simultaneously for computing $X \times W_Q$. In addition, through HGBL and HGBLB, the X-CIM can perform the MAC operation for $S_i = R_i \times X^{\mathrm{T}}$.

B. Dataflow Optimization

In practice, the size of *Matrix X* always exceeds the storage capacity of a CIM bank. *Matrix X* needs to be tiled and allocated to multiple X-CIM banks. For example, each 32-kb X-CIM bank can store 16 rows of *Matrix X* with parameters of INT8 precision and $d_{\mathrm{model}} = 256$. Therefore, a total of 128 input tokens are distributed and stored in the X-CIM array. In CRCIMR, the straightforward dataflow is illustrated in Fig. 5(b), in which each intermediate matrix is generated row-by-row via inner product-based calculations by respective CIM macros. For example, each row of S (S_i) is generated through using rowwise approach as $X_i \times W_Q \times W_K^{\mathrm{T}} \times X^{\mathrm{T}}$. However, this direct dataflow results in massive pipeline stalls.

The X-CIM bank encounters a potential read-compute conflict in the correlative computation related to *Matrix X*. In the row-by-row generation, one X-CIM bank (taking Bank#K as an example) performs $R_i = X_i \times W_Q \times W_K^{\mathrm{T}}$ by configured in Read mode, continuously providing its internally stored input tokens. Then, all X-CIM banks perform $S_i = R_i \times X^{\mathrm{T}}$ by configured in CIM mode for generating the entire elements

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

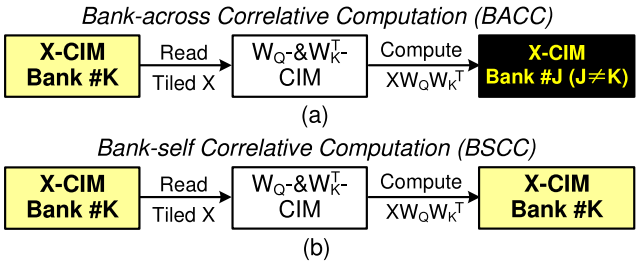


Fig. 7. Correlative computations of AttCIM include: (a) BACC and (b) BSCC.

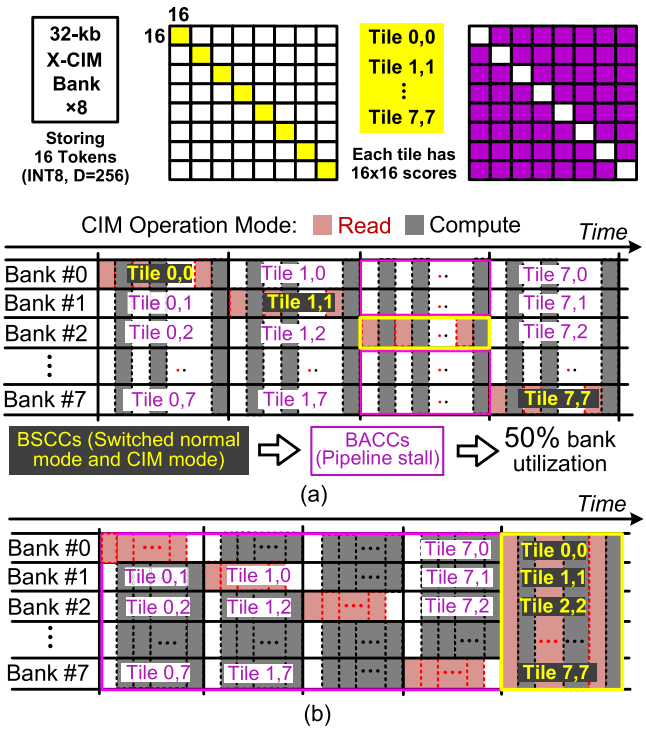


Fig. 8. Operating modes switching of X-CIM banks in (a) traditional dataflow and (b) optimized dataflow.

of S_i gradually. Since each X-CIM bank can only work in one mode, the X-CIM bank that provides input tokens will encounter the read-compute conflict in the generation of *Matrix S*. As a result, when Bank#K works in CIM mode, it cannot provide activations to the W_Q-CIM, thereby resulting in the pipeline stall.

By dataflow optimization, we eliminate the pipeline stall by decoupling the correlative computation across multiply X-CIM banks. The entire correlative computation is separated by two parts, and named by bank-self correlative computation (BSCC), and bank-across correlative computation (BACC). Fig. 7 shows the concept of BSCC and BACC. The BSCC represents the computation that only involves the stored data of a single X-CIM bank. The BACC represents the computation involving different X-CIM banks. Therefore, the X-CIM bank that performs the BSCC experiences the read-compute conflict. Fig. 8 shows the switching of operating modes in eight X-CIM banks. We assume that Bank#K performs the BSCC. In the traditional dataflow, the BACC has to be temporarily stopped when Bank#K switches to CIM mode. Consequently,

Algorithm 1 Optimized Dataflow in CRCIMR

input : Weight W_Q, W_K; Token $X \in \mathbb{R}^{N\times D}$; attention span N_L; X-CIM bank count Num_{Bank}; Tokens storing in Bank#K $X_{Bank\#K}$, $K = 1, \ldots, Num_{Bank}$

output: *Matrix Score (S)*

1 *// Performing BACCs in advance*;
2 **for** $j \leftarrow 0$ **to** Num_{Bank} **do**
3 **for** $i \leftarrow 0$ **to** N_L/Num_{Bank} **do**
4 $R_{i+\frac{j\cdot N_L}{Num_{Bank}}} = X_{i+\frac{j\cdot N_L}{Num_{Bank}}} \times W_Q \times W_K^{\mathrm{T}}$;
5 $S_{i+\frac{j\cdot N_L}{Num_{Bank}},\ BACC} = R_{i+\frac{j\cdot N_L}{Num_{Bank}}} \times X^{\mathrm{T}}_{Bank\#K,K\neq j}$
6 *// Performing BSCCs when completing BACCs*;
7 **for** $i \leftarrow 0$ **to** N_L/Num_{Bank} **do**
8 **for** $j \leftarrow 0$ **to** Num_{Bank} **do**
9 $S_{i+\frac{j\cdot N_L}{Num_{Bank}},\ BSCC} = R_{i+\frac{j\cdot N_L}{Num_{Bank}}} \times X^{\mathrm{T}}_{Bank\#j}$
10 *// Updating X-CIM quasi-statically for the complete S*;
11 **for** $i \leftarrow 0$ **to** $N - N_L$ **do**
12 Write X_{i+N_L} in X-CIM, replacing X_i;
13 $S_{i+N_L} = X_{i+N_L} \times W_Q \times W_K^{\mathrm{T}} \times X^{\mathrm{T}}_{i+1:i+N_L}$

the read-compute conflict degrades the overall utilization of X-CIM banks down to 50%. Algorithm 1 shows the optimized dataflow to avoid the impact of the read-compute conflict. The main idea is to avoid performing BACCs and BSCCs at the same time. First, the X-CIM array performs BACCs until completion. Then, all X-CIM banks perform BSCCs simultaneously. When performing BACCs, each X-CIM bank is configured to send input tokens to the other seven X-CIM banks in a continuous cycle. Meanwhile, the seven X-CIM banks perform BACCs simultaneously. After BACCs, each X-CIM bank receives different rows of *Matrix R* to perform BSCC simultaneously, generating eight tiles with 16×16 elements of *Matrix S* that distributed in the diagonal. Fig. 8(b) shows the operating mode in the optimized dataflow. To avoid the read-compute conflict, all BACCs are processed in a fully pipeline. We analyze the improvement of X-CIM utilization in theory. We assume that each X-CIM bank performs BSCCs while working in Read mode during k cycles. Thus, the X-CIM array consumes $2k$ cycles with 50% utilization in BSCCs. When the X-CIM array has n CIM banks, it consumes $n \cdot k$ cycles with $(n-1)/n$ utilization in BACCs. Therefore, separating BACCs and BSCC can improve the overall X-CIM utilization to $n/(n+2)$.

When processing long sequences, the X-CIM array typically cannot store all input tokens. We achieve a quasi-static token loading by leveraging the local sparse attention. To define attention span without loss of generality, we assume that each input token only effectively attends to a total of N_L tokens adjacent to its left and right. Due to this attention sparsity, only a different token involves the generation of two adjacent rows of *Matrix S*, as shown in Fig. 9. Therefore, the X-CIM array stores the new token into the memory address of the earliest attended token, as shown in Line 11 of Algorithm 1.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

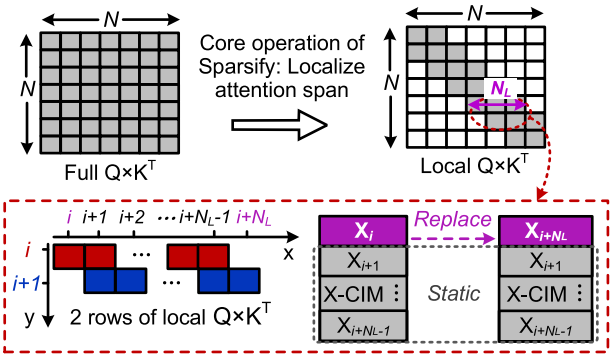


Fig. 9. Quasi-static data loading in the X-CIM array.

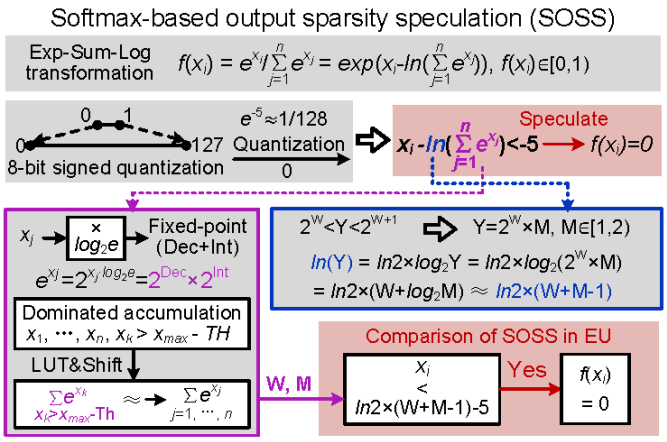


Fig. 10. Principle of Softmax-based output sparsity speculation.

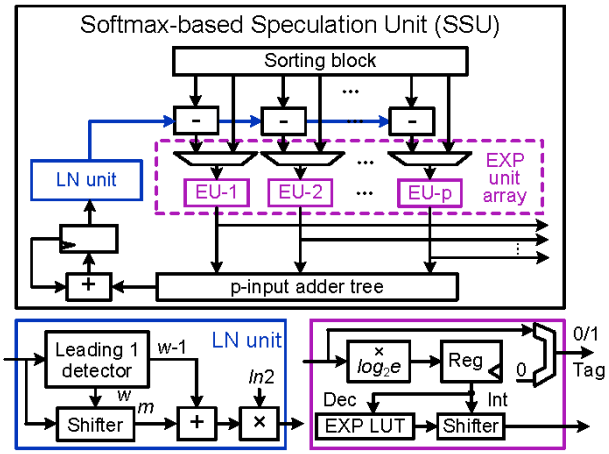


Fig. 11. Architecture of the SSU.

V. Softmax-Based Output Sparsity Speculation

Essentially, the attention mechanism is the search for content-based similarity. Each row of attention scores is normalized by the Softmax function to generate attention probabilities, indicating the relevance of attended tokens. Although the attention span has been localized, many attended tokens may be still irrelevant to the current Q. Through Softmax function and quantization, the number of zero-valued attention probabilities increases to 10× compared to the number of zero-valued scores. The widely distributed output sparsity implies massive redundant computations during the computing of $Q \times K^{\mathrm{T}}$. However, we face three challenges to utilize the output sparsity. First, dynamic sparsity cannot be pre-determined before inference, and its distribution varies drastically. Second, different from the ReLU sparsity, which explicitly outputs zero-valued activations, the Softmax induces implicit sparsity, where all positions are assigned non-zero values. Third, the previous speculation methods focus on ReLU sparsity, which suffers efficiency degradation due to the complex logic of Softmax. To address these issues, we propose the Softmax-based output sparsity speculation (SOSS) method to predict the attention score sparsity with a series of approximate computations.

A. Principle of SOSS and Architecture of SSU

Fig. 10 shows the SOSS. To speculate sparse outputs efficiently, we adopt the log-sum-exp trick [21], in which the complicated division operation is replaced by a subtraction operation and a natural logarithm operation (LnOp). Through the transformation, each Softmax result is obtained by an exponential composite function. The Softmax function induces implicit sparsity, where outputs less than -5 are quantized to zero in 8-bit quantization. In a word, an score (s_{ij}) is normalized to zero when satisfying that $s_{i,j} - \ln(\sum_{j=0}^{n-1} e^{s_{ij}}) < -5$, where s_{ij} represents the jth attention score in ith row. To perform the comparison efficiently, we also adopt the respective transformations for the natural exponential operation (ExpOp) and the LnOp [22], [23]. First, the ExpOp is transformed into Base-2 exponentiation as follows:

$$e^{s_{ij}} = 2^{\log_2 e^{s_{ij}}} = 2^{s_{ij} \cdot \log_2 e} \tag{3}$$

where the term $s_{ij} \cdot \log_2 e$ is divided into an integer part (Int) and a fractional part (Frac). Equation (3) can be further simplified as follows:

$$e^{s_{ij}} = 2^{\mathrm{Int}} \cdot 2^{\mathrm{Frac}} = 2^{\mathrm{Frac}} \ll \mathrm{Int}. \tag{4}$$

The ExpOp can be performed by computing 2^{Frac} with a shift of 2^{Int}. The SOSS further reduces the amount of ExpOps by a pre-trained threshold TH. If s_{ij} exceeds $s_{i,\max}$-TH, s_{ij} is determined as the score that affects the accumulation result of ExpOps. Second, the positive accumulation result of ExpOps can be represented as the product of a value M within $[1, 2)$ and W power of 2. Thus, the corresponding LnOp result can be approximately obtained by the following equation:

$$\begin{aligned} \ln(Y) &= \ln 2 \times \log_2^Y = \ln 2 \times \log_2\left(2^W \times M\right) \\ &= \ln 2 \times \left(W + \log_2 M\right) \overset{\text{fitting}}{\underset{\text{linear}}{=}} \ln 2 \times (W + M - 1). \end{aligned} \tag{5}$$

As a result, the SOSS determines sparse outputs by the comparison at the bottom right of Fig. 10.

Fig. 11 shows the SSU that executes the SOSS. The SSU consists of a score selector, an ExpOp unit (EU) array, an adder tree, and an LnOp unit (LNU). The score selector uses TH to select the larger scores that affect the accumulation of ExpOps. The EU consists of a constant multiplier for $s_{ij} \cdot \log_2 e$, a shifter for 2^{Int}, a lookup table for 2^{Frac}, and a comparator to generate a sparse tag. The LNU consists of a leading one detector to generate M, W, and a MAC unit to compute the LnOp value.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

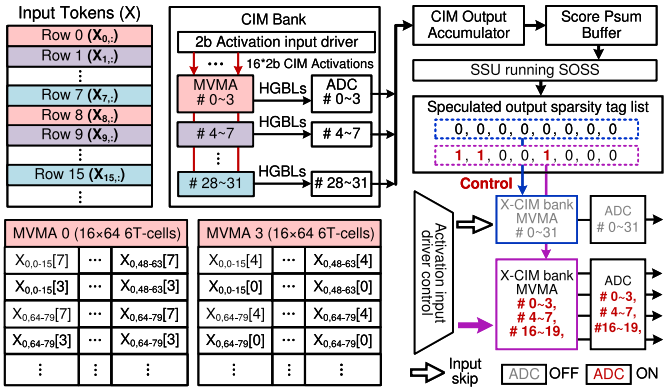


Fig. 12. CRCIMR dataflow with SOSS, in which using high 4-bit input token to speculate sparse outputs.

B. CRCIMR Dataflow With SOSS

The CRCIMR uses a high 4 bit of X (X_{H4B}) to compute the high bits of the attention score, which is used to speculate the zero-valued attention P through the SOSS. Fig. 12 shows the CRCIMR dataflow with the SOSS to eliminate redundant low 4-bit X (X_{L4B}) operations. Before speculating, X_{H4B} and X_{L4B} are split by bit and stored in four adjacent WBCAs with the same column address and the adjacent row address. Each WBCA with 64 b × 16 address spaces stores multiple rows of X, while the elements of an X row are stored on contiguous column addresses. During speculating, the cell that stores X_{H4B} is taken as activation and weight, generating a speculation list through the comparisons of EUs. When the sparse output is speculated, the corresponding list entry is set as 0-entry, otherwise 1-entry. After speculating, eight successive entries are read from the list, enabling the computation of X_{L4B}. If the number of 0's in eight entries is 8, the subsequent computations are redundancies and skipped given an all-zero output. Otherwise, the entry with 1-entry enables the corresponding four WBCAs and four ADCs.

VI. DACIMA Based on Sparse L-Shape Product

Two dynamic MMs, $Q \times K^{\mathrm{T}}$ and $P \times V$, dominate the computing of the MHSA layer. We have detailed the CRCIMR with $Q \times K^{\mathrm{T}}$ decomposition, which entirely fixes the computing of S in memory. However, the CRCIMR introduces pipeline bubbles when performing $P \times V$ decomposition since the decomposed X is multiplied in the opposite direction of the $Q \times K^{\mathrm{T}}$ decomposition. To avoid this issue, we design an rPEA with W_V-CIM, called DACIMA, which performs $P \times V$ in a fully pipeline based on a sparse L-shape product.

A. Motivations of DACIMA and L-Shape Product

Similar to $Q \times K^{\mathrm{T}}$, $P \times V$ is also decomposed into two cascaded MMs as $P \times X \times W_V$, appearing as an inner product with the static weight W_V and the quasi-static input X. However, since SRAM-CIM has a deterministic and single MAC direction, the X-CIM cannot support both MAC operations of $S_i = R_i \times X^{\mathrm{T}}$ and $P \times X$. As a result, to optimize the dynamic MM of P and V in the CIM architecture, additional digital processing elements (PEs) need to be introduced to receive the tokens provided by X-CIM, which assist the W_V-CIM to compute $P \times X \times W_V$. Fig. 13(a) shows the DACIMA that consists of a fully digital rPEA and two analog

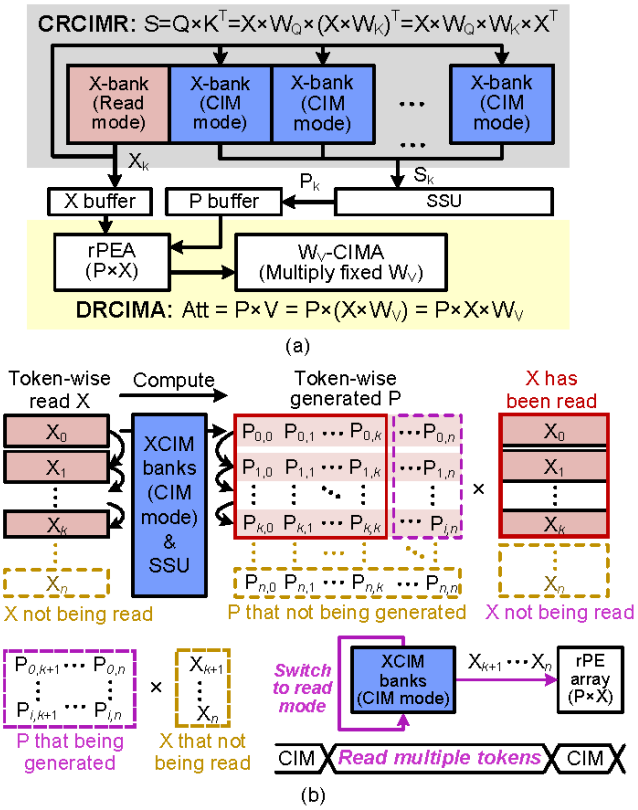


Fig. 13. Motivation of DACIMA. (a) Architecture of DACIMA with V decomposition. (b) Pipeline bubbles of X-CIM based on inner product.

CIM macros that store W_V. The rPEA uses the tokens that read from X-CIM banks to perform $P \times X$. The rPEA generates the effective products row by row, which are sent to the W_V-CIM for multiplying W_V.

Conventionally, the inner product method is used as the meta operation for MMs. However, when the PE performs $P \times X$ based on the inner product, the pipeline bubble is generated in the CRCIMR, as shown in Fig. 13(b). In the CRCIMR, each row of X matrix is read from X-CIM to perform BACCs, generating a complete row of S. When kth row of P (P_k) is generated, $k+1$ rows of X have been read out from the X-CIM. However, since P_k needs to be multiplied by the complete rows of X based on the inner product, the DACIMA has to read the subsequent rows of X from the CRCIMR. As a result, the X-CIM macro working in CIM mode needs to switch to read mode, leading to a large number of pipeline bubbles in the CRCIMR due to limited memory access bandwidth.

B. DACIMA With Sparse L-Shape Product

To avoid the abovementioned pipeline bubbles, the DACIMA utilizes the token that read in the computing of $X \times W_Q$ to compute $P \times X$ based on the sparse L-shape product, as shown in Fig. 14(a). $P_{k;0:k-1}$ is combined with $P_{0:k;k}$ to a vertically symmetrical L-shaped data format. $P_{k;0:k-1}$ represents k elements of the generated P_k, and $P_{0:k;k}$ represents the kth elements of the generated $P_i, i = 0 \sim k$. These L-shaped elements of P are multiplied by the tokens that read in the computing of $X \times W_Q$. Once a set of L-shape elements is generated, the rPEA multiplies it with the buffered tokens intermediately. Since each token attends to only a subset of

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

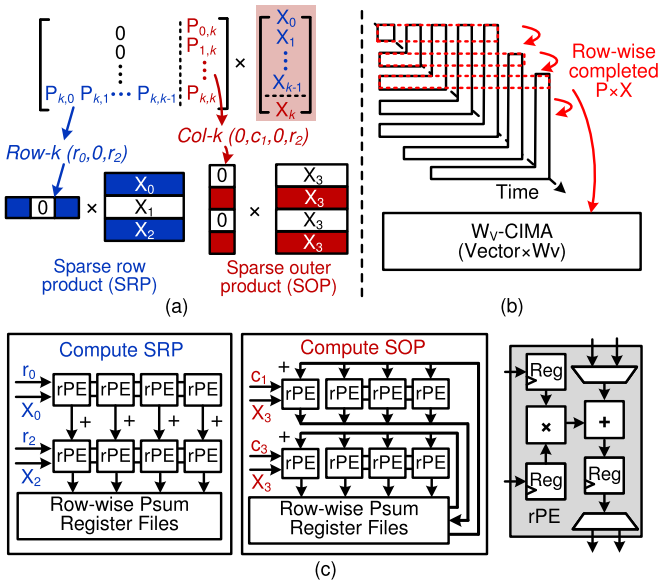


Fig. 14. DACIMA with sparse L-shape product. (a) Sparse L-shape product of $P \times V$. (b) Compute flow of W_V-CIM. (c) Example of L-shape dataflow in rPEA with 2×4 rPEs.

the entire tokens, only the attention possibilities around the diagonal direction of P are effective in the computing of $P \times X$. As the diagonal-dense P is computed with the L-shape sliding, the output matrix of $P \times X$ is completed row by row. As shown in Fig. 14(b), the rowwise generated results of $P \times X$ enable the pipelining of vector-MM in W_V-CIM.

Considering the sparsity of P, the rPEA performs the L-shape product by splitting it into a sparse row product (SRP) and a sparse out product (SOP). For SRP, all the non-zero elements of $P_{k;0:k-1}$ are multiplied with the corresponding rows of X, where the row index of X is determined by the index of the non-zero element of $P_{k;0:k-1}$. The products are accumulated as the partial sums (Psums) of the kth row of the output matrix. For SOP, the non-zero element of $P_{0:k;k}$ is selected to multiply the kth row of X, producing Psums for k rows of the output matrix. By inserting MUXes, the rPE handles two kinds of data transfer and Psums accumulations, respectively, for SRPs and SOPs. Fig. 14(c) shows a 2×4 rPEA as an example. For SRP, the output of each rPE is vertically transferred to the adjacent rPE for SRP accumulations. For SOP, each rPE receives two multipliers and a buffered Psum, and sends the output to the Psums register files for SOP accumulations.

VII. Measurement Results

A. Experimental Setup and Chip Specification

We test AttCIM by a host PC, a field-programmable gate array (FPGA) board, a test printed circuit board (PCB), a dc power supply, and a current waveform analyzer, as shown in Fig. 15. AttCIM is stacked on a test PCB, which forms a test processor and connects to the FPGA board. The dc power supply provides two-way voltages for AttCIM: analog CIM and digital circuits. The current waveform analyzer is used to obtain the average current of AttCIM when it is running inferences. Before measurements, the configuration information, model parameters, and input tokens are prepared in advance. When operating measurement, the host PC sends these data to AttCIM and receives AttCIM's outputs trough

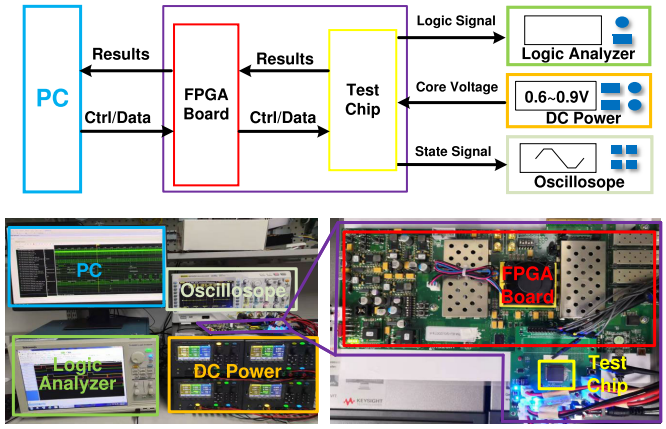
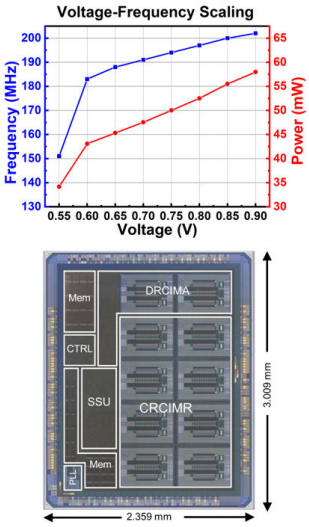


Fig. 15. Test platform of AttCIM.



Metric	This work
Technology	28
Die Area (mm^2)	2.359×3.009
Application	Self-Attention
CIM Size	640 Kb
Digital SRAM Size	166 KB
Activation	INT8
Weight	INT8
Supply Voltage (V)	0.56 – 0.9 *1
Frequency (MHz)	151 – 202 *2
Power (mW)	43.07 – 57.97
Throughput (GOPS)	1243.9 – 1380.3
Energy Efficiency (TOPS/W)	23.81 – 28.88

*1: 0.56-0.9V on digital circuits, 0.9V on CIM macros.
*2: Frequency is the highest frequency measured at digital voltage of 0.56 - 0.9V.

Fig. 16. Chip photograph and summary of AttCIM.

FPGA, obtaining the latency of each inference by recording the start and end times of each run. The average current obtained from the current waveform analyzer is multiplied by the core voltage for power consumption.

Fig. 16 presents the proposed processor's die photograph and performance summary, fabricated in 28-nm CMOS technology with a 7.09-mm^2 area. The chip contains 640-kbit CIM macros and 166-kB normal SRAM. To guarantee the accuracy of analog CIM MAC operation, the supply voltage of CIM macros is always set as 0.9 V. Under 0.56–0.9-V supply voltage for digital modules, the power is 43.07–57.97 mW. The maximum frequency of this processor is 202 MHz, resulting in a peak performance of 1380.3 GOPS in INT8-format computations. The highest system-level energy efficiency is 28.88 TOPS/W at INT8 under 0.56 V, 151 MHz.

Fig. 17 shows the area and power breakdown of AttCIM. It is a system-level CIM processor, integrating multiple CIM macros, adequate on-chip SRAMs, and digital computing logic. The implemented ten CIM macros take over 66.7% area and 60.8% power proportion. The CRCIMR, consisting of eight CIM macros and several accumulation units, takes up 48.8% area and 53.7% power. The SSU accounts for 13.3% area and 26.2% power proportion, in which the sparsity

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

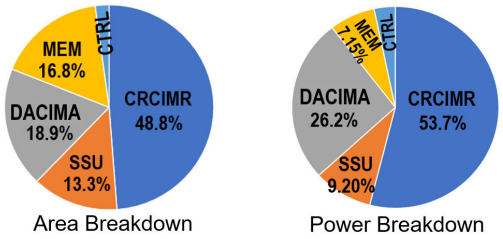


Fig. 17. Area and power breakdown of the proposed processor.

controller only takes 0.86% area and 2.17% power proportion since it only requires simple comparisons. The DACIMA accounts for 18.9% area and 26.2% power, in which the rPE achieves 2.41% speedup by introducing 1.3% additional sparsity-controlling area.

B. Evaluations on Different Transformers

1) Overall Measurement: Table I shows AttCIM's measurement results on two typical Transformer models, BERT-Base [24] and TinyBERT [25], with the WikiHop dataset [26]. We conduct INT8 quantization for the MHSA layer, and test the software and hardware accuracies. AttCIM achieves an accuracy of 70.7% and 74.3% in TinyBERT and BERT-Base, respectively. The accuracy loss only comes from the on-chip error from hardware (0.7% in TinyBERT and 0.6% in BERT-Base) due to the SOSS computation. To evaluate the improvements of the proposed techniques in small CIM chips, we build a simulation system that scales out AttCIM with four chips. Each chip has 128 bidirectional data IO pads. These data IO pads operate with a frequency of 100 MHz. In this simulation setup, DDR3 memory with an actual bandwidth of 8.75 GB/s (70% of theoretical bandwidth) is employed to supply data to these four chips. Each chip is treated as a node, maintaining identical settings as observed in the measurement results obtained from the single-chip AttCIM configuration. The multi-chip system completely stores an attention head's weight matrices and the effective input tokens that need to be attended based on local sparse attention. Off-chip memory is included to evaluate multi-chip system-level consumption in energy and latency but excluded to evaluate chip-level improvement. The baseline is AttCIM performing local attention without exploiting matrix decomposition. In comparison, our work runs MHSA layers with correctly configuring CRCIMR and DACIMA and sets the attention span as 128 tokens. AttCIM achieves 2.01× speedup and 2.24× energy saving on TinyBERT. AttCIM achieves 1.83× speedup and 2.07× energy saving on BERT-Base.

We also evaluate the proposed techniques, respectively. The CRCIMR and the SSU, respectively, optimize the mismatch and redundancy in the computing of $Q \times K^{\mathrm{T}}$, while the DACIMA avoids pipeline bubbles in the computing of $P \times V$. Since each Transformer model is built by stacking multiple attention layers of the same size, we report evaluation results of the CRCIMR and DACIMA (without considering sparsity) on one stack, respectively, as shown in Table II. Based on the CRCIMR and DACIMA, we further provide layerwise evaluation results (with different sparsity) to illustrate the energy/latency saving in the SSU and the sparse L-shape product method.

TABLE I
OVERALL EVALUATION ON TRANSFORMER MODELS

Model	**TinyBERT**	**BERT-Base**
Task; Dataset	Question Answering; WikiHop	
Data Precision	INT8	
Software Accuracy	71.4%	74.9%
Hardware Accuracy	70.7%	74.3%
Execution Time ^{*1,2,3}	0.674 ms	2.022 ms
Energy Consumption ^{*1,2,3}	3.90 mJ	11.71 mJ
Speedup ^{*4,5}	3.01x	2.83x
Energy Saving ^{*4,5}	3.24x	3.07x

∗1: Off-chip DDR3 memory is excluded into latency and energy.
∗2: Measured at 0.9V, 202MHz in digital modules for high-performance evaluation.
∗3: Attention sparsity = 75% with a total of 512 input tokens.
∗4: Off-chip memory access is excluded to focus on chip-level improvement.
∗5: Assuming that the baseline also exploits local sparse attention.

TABLE II
LAYER-TYPE EVALUATION OF THE CRCIMR AND DACIMA

Model	Layer Type	Execution Time ^{*1,2} (ms)			Energy ^{*1,2} (uJ)		
		Baseline	AttCIM	Speedup	Baseline	AttCIM	Saving
BERT-Base	$X \times W$, $Q \times K^T$	3.807	1.723	1.89×	200.62	102.88	1.95×
	$X \times W_V$ $P \times V$	2.220	1.106	2.21×	166.56	69.11	2.41×
Tiny-BERT	$X \times W$, $Q \times K^T$	0.270	0.137	1.97×	18.18	8.91	2.04×
	$X \times W_V$ $P \times V$	0.060	0.028	2.13×	4.61	2.03	2.27×

∗1: Off-chip memory access is excluded to focus on chip-level improvement.
∗2: Assuming that the baseline also exploits local sparse attention.

2) Analysis on CRCIMR: Since each Transformer model is built by stacking multiple attention layers of the same size, we report evaluation CRCIMR on one random stack. The CRCIMR can be configured to perform $Q \times K^{\mathrm{T}}$ in the traditional manner, where the *Matrix K* is loaded from the global buffer to CIM macros. In BERT-base, the multiplication with X^{T} accounts for about 80% of the computation without taking advantage of the attention sparsity, which is the latency bottleneck in the proposed BACC pipeline. Using attention sparsity can effectively solve this problem. When handling 512 input tokens with 75% attention sparsity in the BERT-base model, the latency of different CIM arrays is balanced in the designed BACC pipeline. Due to the BSCC, the total latency for multiplication with X^{T} is only 3.03% longer than that of other computation phases. We also evaluate the energy breakdown across different phases of the CRCIMR, where the multiplications with W_Q, W_K^{T}, and X^{T} account for 22.8%, 27.46%, and 49.66%, respectively. By decomposing the multiplication of K^{T}, the CRCIMR eliminates the K related accessing in both global buffer and CIM macros, thereby achieving 1.89× speedup and 1.95× energy saving on BERT-Base. Additionally, the CRCIMR achieves a 1.95× speedup and 2.04× energy saving on TinyBERT.

Recently, a TranCIM also focuses on the memory access issue when computing $Q \times K^{\mathrm{T}}$ in the CIM architecture. Through TransCIM's pipeline mode in two customized CIM engines (SEngine and DEngine), the DEngine latency involved with the K related writes can be hidden during the K generation of the SEngine. However, the DEngine needs to be

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

TABLE III
LAYERWISE EVALUATION OF SSU AND SPARSE L-SHAPE DATAFLOW

Baseline	Latency (ms)		Energy (uJ)	
CRCIMR	1.723×12		102.88×12	
DACIMA	1.106×12		69.11×12	
Improve	w/ SSU	w/ rPEA	w/ SSU	w/ rPEA
Layer 1	1.11×	1.29×	1.22×	1.25×
Layer 2	1.21×	1.28×	1.33×	1.24×
Layer 3	1.25×	1.24×	1.38×	1.20×
Layer 4	1.24×	1.34×	1.37×	1.34×
Layer 5	1.25×	1.38×	1.39×	1.34×
Layer 6	1.27×	1.36×	1.41×	1.36×
Layer 7	1.35×	1.45×	1.49×	1.32×
Layer 8	1.37×	1.48×	1.51×	1.43×
Layer 9	1.26×	1.66×	1.41×	1.61×
Layer 10	1.25×	1.68×	1.40×	1.63×
Layer 11	1.37×	1.72×	1.53×	1.67×
Layer 12	1.35×	1.57×	1.48×	1.52×
Average	1.27×	1.45×	1.42×	1.44×

configured in Write mode continuously until storing the whole K matrix for performing subsequent $Q \times K^{\mathrm{T}}$. Essentially, since Q and K are both dynamically generated, two CIM engines of TranCIM cannot work in CIM mode at the same time, achieving about 50% of the overall compute utilization. Compared to TranCIM, the whole CIM macros of the CRCIMR only need to store stationary data instead of dynamically generated data, thus potentially working in CIM mode at the same time. However, in CRCIMR, each X-CIM bank needs to switch between Read mode and CIM mode alternatively in the BSCC, reducing the compute utilization of the computational dependent BACCs to 50% again. We perform BACCs and BSCCs in different periods, thereby achieving 1.60× speedup.

3) Analysis on SSU: The SSU captures dynamic sparsity of P from the operation of X_{H4B}. The resulting sparsity index is then used to control the operation of X_{L4B}. Since four adjacent WBCAs' outputs are aggregated into a value, each X-CIM bank that integrated 32 WBCAs generates eight aggregated values in each cycle. The SSU speculates 41.2% elements of P to zero with 0.6% accuracy loss in Bert-Base, in which eight consecutive sparse elements account for 51.6% of the total speculated sparsity. Each zero-valued sparsity-index disables four WBCAs and four ADCs correspondingly, saving unnecessary power consumption. If eight consecutive sparsity indexes are zero-valued, the corresponding computations are skipped given an all-zero outputs directly. As a result, the SSU eliminates 21.3% computations by skipping the computation of the entire X-CIM bank and reduces 18.8% computations by disabling the corresponding WBCAs and ADCs. As depicted in Table III, across different layers, the SSU demonstrates a speedup ranging from 1.11 to 1.37× speedup and energy savings of 1.22–1.67× by skipping the entire CIM bank and disabling WBCAs and ADCs, respectively.

4) Analysis on DACIMA: To fairly evaluate the traditional CIM dataflow for $P \times V$ in DACIMA, the W_V-CIM is configured to load an intermediate V matrix from the global buffer and the rPEA is power-gated. Since memory access

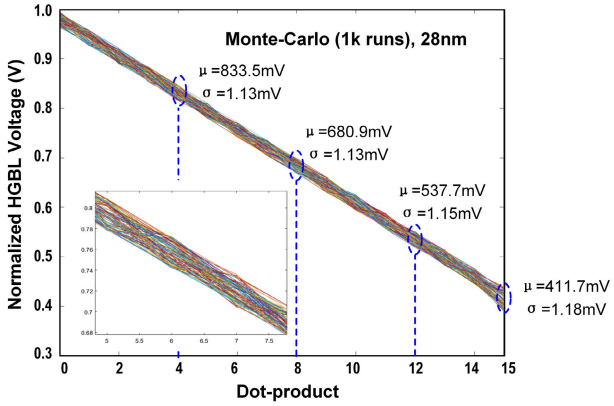


Fig. 18. Transfer function of the CIM macro.

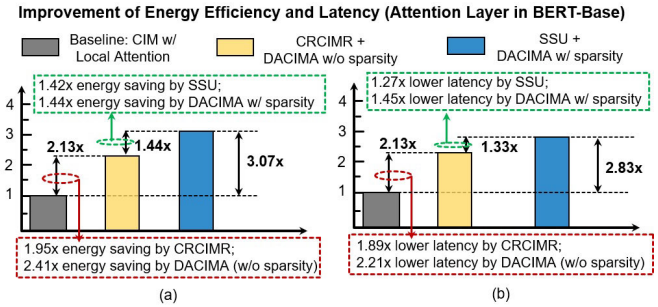


Fig. 19. Technique breakdown analysis on BERT-base. (a) Energy efficiency improvement. (b) Latency saving.

bandwidth is much smaller than computation parallelism, the V-related writes account for 22.3%∼31.1% latency in the computing of $P \times V$. Compared with the traditional manner, $P \times V$ is decomposed into two cascaded multiplication as $P \times X \times W_V$, and performed in rPEA and W_V-CIM, respectively, thereby avoiding the V-related CIM loading. The rPEA performs $P \times X$ based on the sparse L-shape product. Compared with the inner product, the L-shape product only utilizes input tokens that have been read when CRCIMR performs $X \times W_Q$, which avoids the interruption for the computing of $P \times X$. It is the cornerstone to realize a fully pipeline for both $Q \times K^{\mathrm{T}}$ and $P \times V$. Compared with the inner product computation, the CRCIMR achieves 2.21× speedup and 2.41× speedup and energy saving in the BERT-base. The L-shaped elements of P are sparse due to the Softmax function. To further improve energy efficiency, the rPEA computes $P \times V$ based on the sparse rowwise product and outer product, reducing an average of 31.2× computations varying attention layers. Overall, the DACIMA can improve the energy efficiency by 10.8× in the computing of $P \times V$, pushing the system-level energy efficiency to 28.8 TOPS/W at INT8 precision.

5) Analysis on SRAM-CIM Macro: The area of the SRAM-CIM macro is 280×360 μm. The 4-bit ADC accounts for most of the area (39.87%) and energy consumption (52.95%) in CIM mode. The WBCA accounts for the second largest proportion of the area (28.91%) with only 8.78% energy consumption. Each WBCA contains 16 × 64.6T SRAM cells and 16 LCCs. Within a sub-block comprising 16 SRAM cells, the area overhead of the LCC is 1.34× greater than of the compact 6T-SRAM cells. The energy efficiency of the SRAM-CIM macro is 7.87 TOPS/W at 0.9 V,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON WITH THE STATE-OF-THE-ART PROCESSORS

	Wang *et al.* ISSCC'22-29.2 [26]	**Ben *et al.* VLSI'22 [27]**	**TranCIM [9] ISSCC'22-29.3**	**MulTCIM [28] ISSCC'23-16.1**	**Liu *et al.* [29] ISSCC'23-16.2**	**This Work**
Architecture	non-CIM	non-CIM	Digital CIM	Digital CIM	Digital CIM	Analog CIM
Technology	28nm	5nm	28nm	28nm	28nm	28nm
Precision	INT12	INT4/INT8	INT8/16	INT8/16	INT8	INT8
Sparsity Level	Attention	No Reported	Attention	Attention/Token/Bit	Attention/Weight	Attention
Die Area (mm^2)	6.82	0.152	6.83	14.36	3.93	7.098
Supply Voltage (V)	0.56 - 1.1	0.45 - 1.05	0.6 - 1.0	0.6 - 1.0	0.64 - 1.03	0.56 - 0.9
Frequency (MHz)	50 - 510	152 - 1760	80 - 240	85 - 275	20 - 320	80 - 275
Power (mW)	12.06 - 272.8	No Reported	27.04 - 118.2	28.83 - 152.8	8.72 - 250.65	43.07 - 57.97
Peak Performance ^{*1,2} (TOPS)	4.07	1.8 (INT8)	0.74 (INT8)	3.55 (INT8)	3.33	1.38
Area Efficiency ^{*1,2} (GOPS/mm^2)	596.8	11764.7	108.3	247.2	847.3	194.4
System Energy ^{*1,2} Efficiency (TOPS/W)	Q/K/V Gen: 4.25; QK^T & PV: 27.56 ^{*3}	Overall attention: 39.1 (INT8)	Overall attention: 20.5 (INT8) ^{*4}	Overall attention: 101.1 (INT8) ^{*5}	Q/K/V Gen: 4.25 QK^T & PV: 53.83 Overall: 25.22 ^{*6}	CRCIMR: 13.5 DACIMA: 28.8 ^{*7} Overall: 19.38

$*1$: Off-chip memory access is excluded. All on-chip computing time is included.
$*2$: One operation represents 1 addition or 1 multiplication.
$*3$: 90% output sparsity due to 90% weakly related tokens ration on GPT-2 model.
$*4$: Attention sparsity = 93.75% since the attention span is set to 256 for BERT-base model with 4096 input tokens.
$*5$: Attention sparsity = 70%. Modal X's token sparsity = 30%. Modal Y's token sparsity = 50%. Softmax-MSB bit sparsity = 58.03%.
$*6$: Attention sparsity = 92% and weight sparsity = 75% for Enwik-8 task with 1024 input tokens.
$*7$: Attention sparsity = 75% since the attention span is set to 128 for BERT-Base model with 512 input tokens.

202 MHz. The area efficiency of the SRAM-CIM macro is 535.1 GOPS/mm^2. Considering attention sparsity and output sparsity, the SRAM-CIM macro achieves 52.57 TOPS/W on the deployed models. To avoid potential sensing error due to limited sensing margin, we reduce the parallelism of MAC operations to limit the expected dot-product range to "0"–"16." Within this range of MAC value, we can ensure the linearity of analog MAC operations and ensure that the ADC has a sufficient sensing margin to all values. Fig. 18 shows the simulated linearity and process variation using 1k Monte Carlo runs. A row of 16 bit cells is used for the simulation. As each analog current path is added, the expected dot-product result is swept from 0 to 15 with a step size of 1. Based on the Monte Carlo simulation results, we find that the largest standard deviation (σ) is 1.18 mV, which appears when the dot-product is 15. The largest standard deviation is 3.31% of the full range of HGBL voltages in this case. After measurement, each CIM macro of the chip can maintain the same output results as the expected computation result in the tested dataset.

6) Technique Breakdown Analysis: Fig. 19 shows a technique breakdown analysis to clearly illustrate how the energy and latency are saved with AttCIM's features, which further explains the data relationship between Tables I–III. The baseline is implemented on AttCIM with local attention but without matrix decomposition. Under the BERT-base model, the CRCIMR achieves 1.89× speedup and 1.95× energy saving in the computing of $X \times W_{Q,K}$ and $Q \times K^{\mathrm{T}}$, while the DACIMA achieves 2.21× speedup and 2.41× energy saving in the computing of $X \times W_V$ and $P \times V$. The two techniques reduce to 2.13× energy consumption and latency for the entire attention layers. When exploiting the dynamic sparsity of *Matrix P*, the CRCIMR achieves 1.27× speedup and 1.42× energy saving, while the DACIMA achieves 1.45× speedup and 1.44× energy saving. The two techniques reduce to 1.44× energy consumption and 1.33× latency for the entire attention layers. Finally, AttCIM reduces to 3.07× energy consumption and 2.83× latency compared to the baseline, as listed in Table I.

C. Compared With the State-of-the-Art Processor

Table IV shows a detailed comparison with the state-of-the-art Transformer accelerators. ISSCC'22-29.2 [27] is a digital Transformer accelerator with approximation computing techniques for weak attention. VLSI'22 [28] is a 5-nm digital Transformer accelerator with per-vector scaled quantization. ISSCC'22-29.3 [10] is a digital TranCIM with pipeline/parallel reconfigurable modes. ISSCC'23-16.1 [29] is a digital CIM-based accelerator for multimodal Transformers exploiting attention-token-bit hybrid sparsity. ISSCC'23-16.2 [30] is a digital TranCIM with unstructured weight pruning and local attention. These works exploit either attention sparsity or advanced quantization to improve energy efficiency. Since the tested models and tasks involve different levels of sparsity, it is not appropriate to directly compare the energy efficiency of different works. Here, we only compare our work with TranCIM under the equivalent 75% attention sparsity.

Wang et al. [27] achieve 4.25-TOPS/W energy efficiency in generation of *Matrix Q*, *K*, and *V*. This chip also achieves 27.56-TOPS/W energy efficiency in the computing of $Q \times K^{\mathrm{T}}$ and $P \times V$, when assuming 90% weakly related tokens on INT12 GPT-2 model with 512 input tokens. For a fair comparison, we convert the overall energy efficiency of the digital Transformer accelerator in the generation of *Matrix Q*, *K*, and *S* with 75% attention sparsity based on the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

involved operation count. The converted energy efficiency is 5.94 TOPS/W. Furthermore, we convert the overall energy efficiency of an attention head, which is 6.33 TOPS/W.

MulTCIM [29] achieves 101.1-TOPS/W peak energy efficiency for INT8 data precision with exploiting attention-token-bit hybrid sparsity (attention sparsity = 70%, token sparsity = 40%, and Softmax-MSB bit sparsity = 58.03%). Compared to our work, MulTCIM exploits more comprehensive sparsity. This is because there is still massive attention sparsity between the input tokens that have not been removed in the target benchmark (ViLBERT-base). Since MulTCIM does not offer separate energy efficiency data for different phases of the attention layer and only provides the energy efficiency for the entire attention with comprehensive sparsities, we are unable to accurately extrapolate the overall energy efficiency to the scenario with 75% attention sparsity.

TranCIM [10] achieves 20.5-TOPS/W energy efficiency by exploiting a static attention sparsity on INT8 BERT-base. Since TranCIM sets the attention span to 256 with a token length of 4096, the involving attention sparsity is 256/4096 = 93.75%. Similar to MulTCIM [29], TranCIM does not offer separate energy efficiency data for different phases of the attention layer and only provides the energy efficiency for the entire attention. As a result, we are unable to accurately extrapolate the overall energy efficiency to the scenario with 75% attention sparsity. To compare with TranCIM, we assume to optimize 93.75% attention sparsity in AttCIM, thereby achieving 35.17 TOPS/W. Therefore, AttCIM achieves 1.72× energy efficiency improvement.

Liu et al. [30] achieve 11.83-TOPS/W energy efficiency in the generation of *Matrix* Q, K, and V by exploiting 75% weight sparsity. This chip also achieves 53.83-TOPS/W energy efficiency in the computing of $Q \times K^{\mathrm{T}}$ and $P \times V$ by exploiting 92% attention sparsity. Based on the involved operation count, we convert the overall energy efficiency in the generation of *Matrix* Q, K, and S, which is 13.18 TOPS/W. Further, we convert the overall energy efficiency of an attention head, which is 13.57 TOPS/W.

AttCIM achieves an energy efficiency of 13.5 and 28.8 TOPS/W for the CRCIMR and DACIMA, respectively. It achieves 19.38-TOPS/W energy efficiency for the overall attention layer. Thanks to the CIM technique, the CRCIMR achieves a 2.27× system-level energy efficiency improvement compared to [27] in the generation of *Matrix* Q, K, and S. For the entire attention layer, AttCIM achieves 1.43× energy efficiency improvement over [30]. MulTCIM and TranCIM adopt a similar technique to exploit attention sparsity. When using the same ratio of attention sparsity, AttCIM achieves 1.72× energy efficiency improvement over TranCIM [10]. In summary, if the proposed work uses the same sparsity values, this work will achieve similar or better energy efficiency, which results from two reasons. First, instead of writing K matrices in TranCIM's pipeline mode, AttCIM eliminates the CIM loading of K matrices directly, making the CRCIMR stay in CIM mode to perform pipelined $Q \times K^{\mathrm{T}}$ with stationary weights and quasi-stationary tokens. Second, TranCIM designs bitline-transpose-CIM structure to store K^{T} for aligning the input (Q) feeding and weight (K) writing directions in $Q \times K^{\mathrm{T}}$. However, the computation direction conflict between $Q \times K^{\mathrm{T}}$ and $P \times V$ is ignored, TranCIM has to write out V and then load it back into CIM macros in a transpose way, resulting in high energy consumption and latency. AttCIM attends this conflict and solves it by changing the multiplication direction. Instead of performing the inner product, the DACIMA fully utilizes the read tokens to perform $P \times X \times W_V$ based on the L-shape product.

VIII. Conclusion

The attention mechanism introduces dynamic MMs, including $Q \times K^{\mathrm{T}}$ and $P \times V$, which are the computational bottleneck of the Transformer model and do not match the CIM paradigm. In addition, the attention mechanism inherently contains a large number of redundant connections, introducing inefficient computations in dynamic MMs. This article proposes a CIM-based Transformer processor that fabricated in 28-nm CMOS technology, solving several new challenges of accelerating the attention mechanism with three design features. First, the CRCIMR performs the decomposed $Q \times K^{\mathrm{T}}$ in the pipeline with stationary weights and quasi-stationary inputs, avoiding the loading of dynamically generated matrices in the SRAM-CIM. Second, the SSU speculates the output sparsity of the Softmax-activated attention score by high bits of input tokens, reducing overall 18.8% computations of the CRCIMR. Third, the DACIMA performs the decomposed $P \times V$ based on a sparse L-shape product, allowing CIM macros to work in CIM mode to the maximum extent possible. With all these optimization techniques, the measurement results show that this processor achieves 28.88-TOPS/W system-level energy efficiency.

References

[1] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, no. 11, 2017, pp. 6000–6010.

[2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," 2019.

[3] Y. Wang et al., "An energy-efficient transformer processor exploiting dynamic weak relevances in global attention," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 227–242, Jan. 2023.

[4] X. Si et al., "A local computing cell and 6T SRAM-based computing-in-memory macro with 8-b MAC operation for edge AI chips," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2817–2831, Sep. 2021.

[5] J.-W. Su et al., "16.3 A 28 nm 384 kb 6T-SRAM computation-in-memory macro with 8 b precision for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 250–252.

[6] Y.-D. Chih et al., "16.4 an 89 TOPS/W and 16.3 TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.

[7] H. Jia et al., "15.1 a programmable neural-network inference accelerator based on scalable in-memory computing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 236–238.

[8] J. Yue et al., "15.2 A 2.75-to-75.9 TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 238–240.

[9] R. Guo et al., "15.4 A 5.99-to-691.1 TOPS/W tensor-train in-memory-computing processor using bit-level-sparsity-based optimization and variable-precision quantization," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 242–244, doi: 10.1109/ISSCC42613.2021.9365989.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.

[10] F. Tu et al., "A 28 nm 15.59 μJ/token full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 466–468.
[11] R. Guo et al., "A 28 nm 57.6 TOPS/W attention-based NN processor with correlative computing-in-memory ring and dataflow-reshaped digital-assisted Computing-in-Memory array," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, Nov. 2022, pp. 1–3.
[12] R. Guo et al., "TT@CIM: A tensor-train in-memory-computing processor using bit-level-sparsity optimization and variable precision quantization," *IEEE J. Solid-State Circuits*, vol. 58, no. 3, pp. 852–866, Mar. 2023.
[13] Y. Ren, B. Wang, L. Shang, X. Jiang, and Q. Liu, "Exploring extreme parameter compression for pre-trained language models," 2022, *arXiv:2205.10036*.
[14] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," 2019, *arXiv:1904.10509*.
[15] A. Roy, M. Saffar, A. Vaswani, and D. Grangier, "Efficient content-based sparse attention with routing transformers," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 53–68, Feb. 2021.
[16] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. P. Lillicrap, "Compressive transformers for long-range sequence modelling," 2019, *arXiv:1911.05507*.
[17] J. Qiu, H. Ma, O. Levy, W.-T. Yih, S. Wang, and J. Tang, "Blockwise self-attention for long document understanding," in *Proc. Findings Assoc. Comput. Linguistics, Empirical Methods Natural Lang. Process.*, 2020, pp. 2555–2565.
[18] M. Zaheer et al., "Big Bird: Transformers for longer sequences," in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 17283–17297.
[19] P. J. Liu et al., "Generating Wikipedia by summarizing long sequences," 2018, *arXiv:1801.10198*.
[20] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020, *arXiv:2004.05150*.
[21] B. Yuan, "Efficient hardware architecture of softmax layer in deep neural network," in *Proc. 29th IEEE Int. Syst. Chip Conf. (SOCC)*, Nov. 2016, pp. 323–326.
[22] M. Wang, S. Lu, D. Zhu, J. Lin, and Z. Wang, "A high-speed and low-complexity architecture for softmax function in deep learning," in *Proc. IEEE Asia–Pacific Conf. Circuits Syst. (APCCAS)*, Oct. 2018, pp. 223–226.
[23] Y. Zhang et al., "Base-2 softmax function: Suitability for training and efficient hardware implementation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 9, pp. 3605–3618, Sep. 2022.
[24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2019, *arXiv:1810.04805*.
[25] X. Jiao et al., "TinyBERT: Distilling BERT for natural language understanding," 2019, *arXiv:1909.10351*.
[26] J. Welbl, P. Stenetorp, and S. Riedel, "Constructing datasets for multi-hop reading comprehension across documents," 2017, *arXiv:1710.06481*.
[27] Y. Wang et al., "A 28 nm 27.5 TOPS/W approximate-computing-based transformer processor with asymptotic sparsity speculating and out-of-order computing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.
[28] B. Keller et al., "A 17–95.6 TOPS/W deep learning inference accelerator with per-vector scaled 4-bit quantization for transformers in 5nm," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 16–17.
[29] F. Tu et al., "16.1 MulTCIM: A 28 nm 2.24μJ/token attention-token-bit hybrid sparse digital CIM-based accelerator for multimodal transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 248–250.
[30] S. Liu et al., "16.2 A 28 nm 53.8 TOPS/W 8 b sparse transformer accelerator with in-memory butterfly zero skipper for unstructured-pruned NN and CIM-based local-attention-reusable engine," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 250–252.
[31] S. Elfwing, E. Uchibe, and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," 2017, *arXiv:1702.03118*.
[32] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," 2023, *arXiv:1606.08415*.



Ruiqi Guo (Graduate Student Member, IEEE) received the B.S. degree from the School of Microelectronics, Xidian University, Xi'an, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits, Tsinghua University, Beijing, China.

His current research interests include SRAM-based compute-in-memory (CIM), computer architecture and VLSI design, and hardware–software co-design for artificial intelligence (AI) accelerators.



Zhiheng Yue received the B.S. degree in electronic science and technology from Beijing University of Posts and Telecommunications, Beijing, China, in 2017, and the M.S. degree in electrical and computer engineering from the University of Michigan, Ann Arbor, MI, USA, in 2019. He is currently pursuing the Ph.D. degree in electronic science and technology with Tsinghua University, Beijing.

His current research interests include deep learning, compute-in-memory (CIM), artificial intelligence (AI) accelerator, and VLSI design.



Yang Wang received the B.S. degree in electronic science and technology from Xidian University, Xi'an, China, in 2014, and the Ph.D. degree in microelectronics and solid-state electronics from the Institute of Microelectronics, Chinese Academy of Sciences, Beijing, China, in 2019.

He is currently an Assistant Researcher with the School of Integrated Circuits, Tsinghua University, Beijing. His research interests include VLSI of digital signal processing, deep learning, and neural network acceleration.



Hao Li received the B.S. degree in microelectronic science and engineering from Xi'an Jiaotong University, Xi'an, China, in 2019, and the M.S. degree from the School of Integrated Circuits, Tsinghua University, Beijing, China, in 2022.

His current research interests include neural network accelerator, compute-in-memory (CIM), and VLSI design.



Te Hu received the B.S. degree from Tongji University, Shanghai, China, in 2019, and the M.S. degree from the School of Integrated Circuits, Tsinghua University, Beijing, China, in 2022.

His research interests include the architecture design of deep learning systems and compute-in-memory (CIM).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.



Yabing Wang received the B.S. degree from the School of Microelectronics, Xidian University, Xi'an, China, in 2018, and the M.S. degree from the School of Integrated Circuits, Tsinghua University, Beijing, China, in 2022.

His current research interests include memory design and compute-in-memory (CIM).



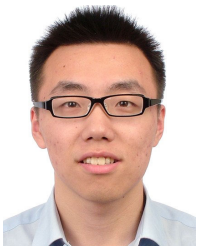
Hao Sun received the B.S. degree from the School of Computer Science and Technology, Tianjin University of Science and Technology, Tianjin, China, in 2011, and the M.S. degree from Beihang University, Beijing, China, in 2018.

He is currently a Senior Engineer with Tsinghua University, Beijing. His research interests include deep learning and machine learning.



Jeng-Long Hsu received the B.S. and M.S. degrees from the Department of Chemistry, National Tsing Hua University (NTHU), Hsinchu, in 1990 and 1993, respectively.

He worked with TSMC, SoNix Technology, and RICHTEK, Hsinchu, from 1999 to 2014. He has more than 20 years of experience in chip design and team leadership. He is currently a Business Development Senior Manager and a Team Leader of analog circuits design with NeoNexus Pte. Ltd., Singapore. His main work covers processing-in-memory (PIM)-related circuit design.



Yaojun Zhang received the B.S. degree from Shanghai Jiao Tong University, Shanghai, China, in 2008, and the M.S. and Ph.D. degrees from the University of Pittsburgh, Pittsburgh, PA, USA, in 2014.

He currently serves as a Design Team Leader with Pimchip Technology, Beijing, China. He has authored about 50 technical publications and contributed to processing-in-memory (PIM) VLSI design with digital SRAM-based circuit structures and system-on-chip (SoC) systems based on SRAM PIM. His current work is developing products using SRAM PIM with low power, small area, and high computing power. And the product goals are to provide users with an advanced artificial intelligence (AI) hardware acceleration solution.



Bonan Yan (Member, IEEE) received the B.S. degree from Beihang University, Beijing, China, in 2014, the M.S. degree from the University of Pittsburgh, Pittsburgh, PA, USA, in 2017, and the Ph.D. degree from Duke University, Durham, NC, USA, in 2020.

He is currently an Assistant Professor with the Institute for Artificial Intelligence, Peking University, Beijing, where he is also a Boya Young Fellow. He has authored about 50 technical publications and contributed to compute-in-memory (CIM) VLSI design with digital SRAM-based circuit structures and analog spike-based memristor-based computing schemes. His research interests include memory-centric processor architectures and artificial general intelligence computing hardware design.



Leibo Liu (Senior Member, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 1999, and the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, in 2004.

He is currently a Professor with the School of Integrated Circuits, Tsinghua University. His research interests include reconfigurable computing, mobile computing, and VLSI digital signal processing (DSP). His research interests include deep learning and machine learning.



Ru Huang received the B.S. (Hons.) and M.S. degrees in electronic engineering from Southeast University, Nanjing, China, in 1991 and 1994, respectively, and the Ph.D. degree in microelectronics from Peking University, Beijing, China, in 1997.

Since 1997, she has been a Faculty Member with Peking University, where she is currently the Boya Chair Professor. She is an Elected Academician with the Chinese Academy of Sciences. She has authored or co-authored five books, five book chapters, and more than 300 articles, including more than 100 articles in IEDM, VLSI Technology Symposium, IEEE ELECTRON DEVICE LETTERS, and IEEE TRANSACTIONS ON ELECTRON DEVICES. She has delivered over 50 keynote/invited talks at conferences and seminars. She holds over 300 patents, including 49 U.S. patents. Her research interests include nano-scaled CMOS devices, ultra-low-power new devices for neuromorphic computing, emerging memory technology, and device variability/reliability.



Shaojun Wei (Fellow, IEEE) was born in Beijing, China, in 1958. He received the Ph.D. degree from the Faculte Polytechnique de Mons, Mons, Belgium, in 1991.

He became a Professor with the Institute of Microelectronics, Tsinghua University, Beijing, in 1995. His main research interests include VLSI system-on-chip (SoC) design, electronic design automation (EDA) methodology, and communication application-specific integrated circuit (ASIC) design.

Dr. Wei is also a Senior Member of the Chinese Institute of Electronics (CIE).



Shouyi Yin (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2000, 2002, and 2005, respectively.

He was a Research Associate with Imperial College London, London, U.K. He is currently a Full Professor and the Vice-Director of the School of Integrated Circuits, Tsinghua University. He has authored more than 100 journal articles and more than 50 conference papers. His research interests include reconfigurable computing, artificial intelligence (AI) processors, and high-level synthesis.

Dr. Yin has served as a Technical Program Committee Member of the top VLSI and electronic design automation (EDA) conferences, such as Asian Solid-State Circuits Conference (A-SSCC), IEEE/ACM International Symposium on Microarchitecture (MICRO), Design Automation Conference (DAC), International Conference on Computer-Aided Design (ICCAD), and Asia and South Pacific Design Automation Conference (ASP-DAC). He is also an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS (TCAS-I), *ACM Transactions on Reconfigurable Technology and Systems* (TRETS), and *Integration, the VLSI Journal*.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:22:26 UTC from IEEE Xplore. Restrictions apply.